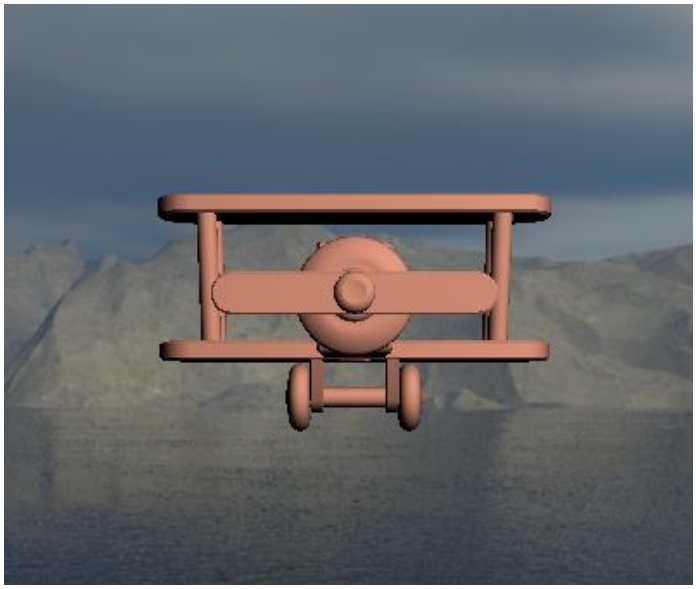
CS7GV5 - Real-Time Animation
Student Name: Stephen Komolafe
Student Number: 21336975

Assignment #01: Plane Rotation

Core Features

Aircraft model with Euler-angle rotation control

- Plane model rendered in scene:

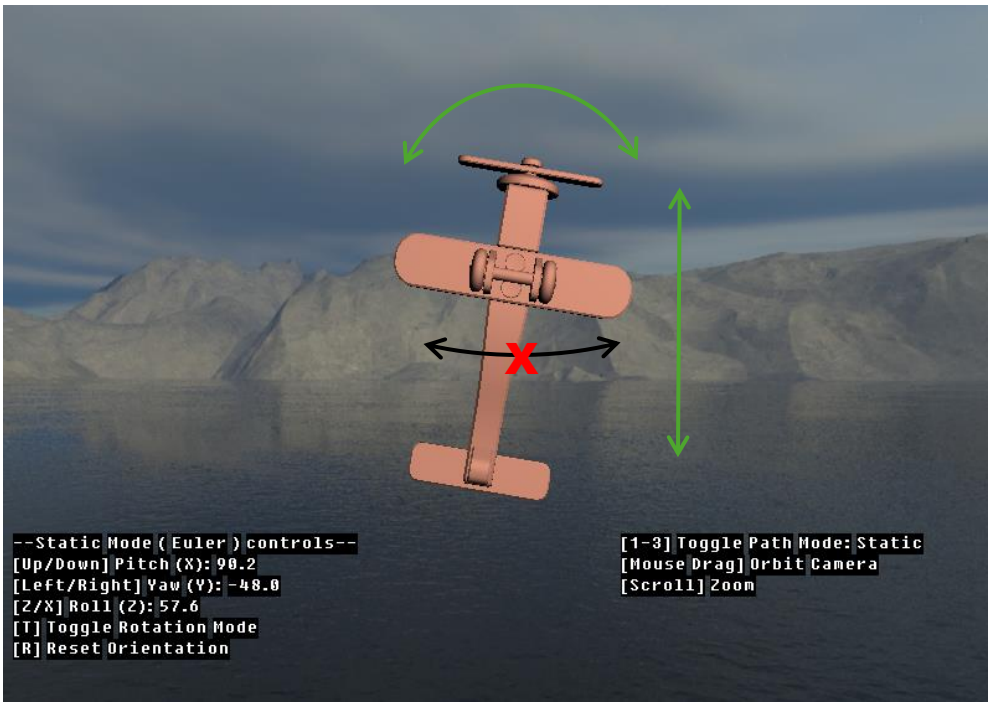


(Plane Model Source: https://skfb.ly/oqoSL)

```
struct ModelData {
    size_t mPointCount = 0;
    std::vector<vec3> mVertices;
    std::vector<vec3> mNormals;
    std::vector<vec2> mTextureCoords;
};

ModelData mesh_data;
Shader* modelShader = nullptr;
const char* PLANE_MODEL = "models/plane.dae";

ModelData load_mesh(const char* file_name) {
    ModelData modelData;
    const aiScene* scene = aiImportFile(
        file_name,
        aiProcess_Triangulate | aiProcess_PreTransformVertices
    );

    if (!scene) {
        fprintf(stderr, "ERROR: reading mesh %s\n", file_name);
        return modelData;
    }

    printf("Loaded: %i meshes\n", scene->mNumMeshes);
    for (unsigned int m_i = 0; m_i < scene->mNumMeshes; m_i++) {
        const aiMesh* mesh = scene->mMeshes[m_i];
        modelData.mPointCount += mesh->mNumVertices;
        for (unsigned int v_i = 0; v_i < mesh->mNumVertices; v_i++) {
            if (mesh->HasPositions()) {
                const aiVector3D* vp = &(mesh->mVertices[v_i]);
                modelData.mVertices.push_back(vec3(vp->x, vp->y, vp->z));
```

```
            }
            if (mesh->HasNormals()) {
                const aiVector3D* vn = &(mesh->mNormals[v_i]);
                modelData.mNormals.push_back(vec3(vn->x, vn->y, vn->z));
            }
            if (mesh->HasTextureCoords(0)) {
                const aiVector3D* vt = &(mesh->mTextureCoords[0][v_i]);
                modelData.mTextureCoords.push_back(vec2(vt->x, vt->y));
            }
        }
    }
    aiReleaseImport(scene);
    return modelData;
}
```

- The plane rotates using pitch, roll, and yaw controlled via keyboard input. Paired with Euler-based rotation gimbal lock* can be demonstrated:



(*When pitch reaches ≈90°, the yaw and roll axes align, resulting in the loss of one degree of freedom.)

```
float pitch = 0.0f; // X-axis
float yaw = 0.0f; // Y-axis
float roll = 0.0f; // Z-axis
(...)
void key_callback(GLFWwindow* window, int key, int scancode, int action, int mods) {
    const float rotSpeed = 60.0f * 0.016f;
    float dPitch = 0.0f, dYaw = 0.0f, dRoll = 0.0f;

    if (key == GLFW_KEY_UP)    dPitch += rotSpeed;
    if (key == GLFW_KEY_DOWN)  dPitch -= rotSpeed;

    if (key == GLFW_KEY_LEFT)  dYaw += rotSpeed;
    if (key == GLFW_KEY_RIGHT) dYaw -= rotSpeed;

    if (key == GLFW_KEY_Z)     dRoll += rotSpeed;
    if (key == GLFW_KEY_X)     dRoll -= rotSpeed;

    pitch += dPitch;
    yaw += dYaw;
    roll += dRoll;
```

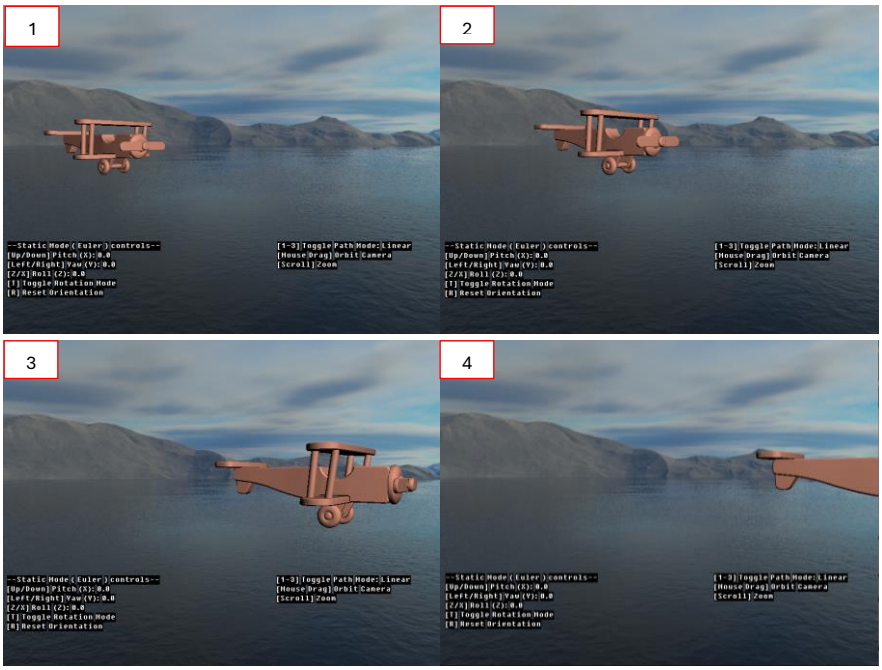
Rotation UI feedback

- On-screen UI shows the current input angles of the planes pitch, roll, and yaw with the ability to reset its orientation:


```
#define GLT_IMPLEMENTATION
#include "gltext.h"
GLTtext* txtControlsL;
GLTtext* txtControlsR;
char controlsBufferL[256];
char controlsBufferR[256];
(…)
void key_callback(GLFWwindow* window, int key, int scancode, int action, int
mods) {
       (…)
       if (key == GLFW_KEY_R && action == GLFW_PRESS) {
              pitch = yaw = roll = 0.0f;
(…)
    snprintf(controlsBufferL, sizeof(controlsBufferL),
        "--Static Mode ( %s ) controls--\n"
        "[Up/Down] Pitch (X): %.1f\n"
        "[Left/Right] Yaw (Y): %.1f\n"
        "[Z/X] Roll (Z): %.1f\n"
        "[T] Toggle Rotation Mode\n"
        "[R] Reset Orientation\n",
        rotModeName,
        displayPitch, displayYaw, displayRoll
    );

    snprintf(controlsBufferR, sizeof(controlsBufferR),
        "[1-3] Toggle Path Mode: %s\n"
        "[Mouse Drag] Orbit Camera\n"
        "[Scroll] Zoom\n",
        pathModeName
    );

    gltSetText(txtControlsL, controlsBufferL);
    gltSetText(txtControlsR, controlsBufferR);

    gltBeginDraw();

    gltColor(1.0f, 1.0f, 1.0f, 1.0f);

    gltDrawText2D(txtControlsL, 10.0f, height - 170.0f, 1.0f);
    gltDrawText2D(txtControlsR, width - 300.0f, height - 170.0f, 1.0f);

    gltEndDraw();
```

Keyframed flight path animation

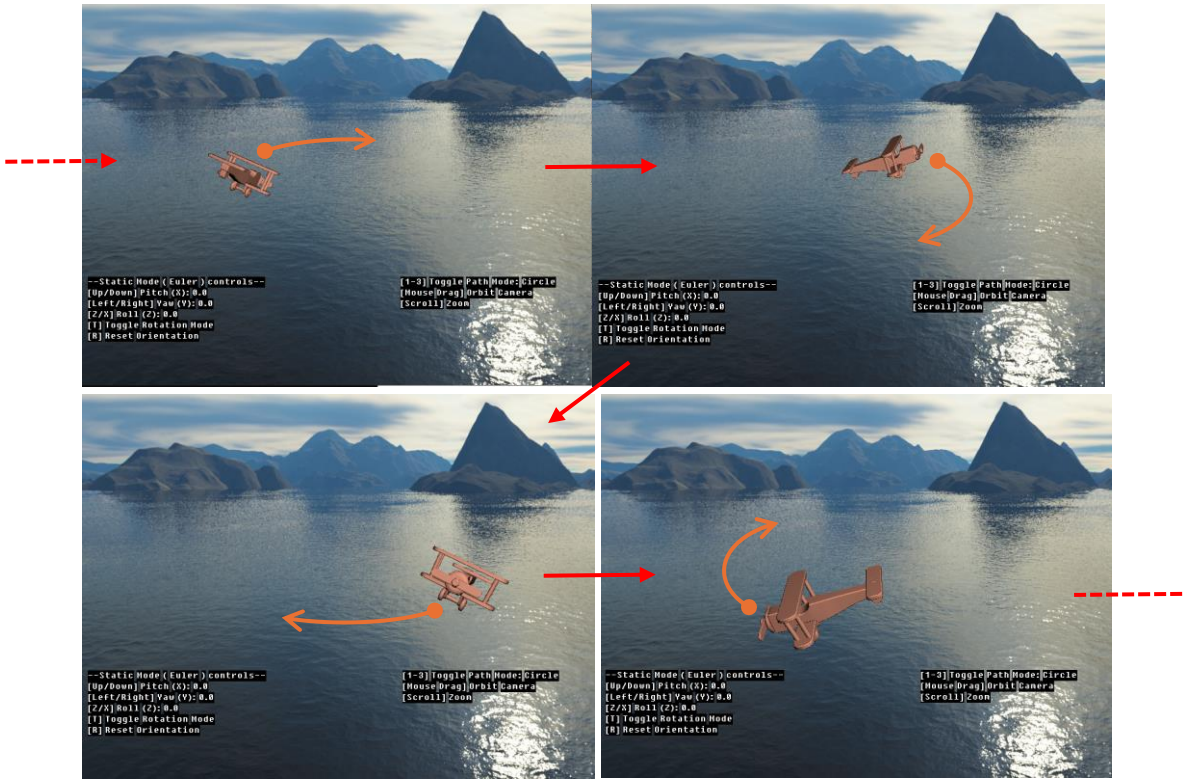
- Implemented a linear keyframed path for the plane to follow:




```
std::vector<Keyframe> flightPath = {
    { glm::vec3(0.0f,  -0.25f,  15.0f), 0.0f },
    { glm::vec3(0.1f,  0.25f,  5.0f), 3.0f },
    { glm::vec3(0.0f,  -0.25f, -5.0f), 6.0f },
    { glm::vec3(-0.1f,  0.25f,  -15.0f), 9.0f }
      };
```

1
2
3
4

- Implemented a circular parametric based path for the plane to follow (plane banking along the trajectory also added):



```
glm::vec3 circlePath(float t) {
    float radius = 12.0f;
    float height = 0.0f;
    float x = radius * cos(t);
    float z = radius * sin(t);
    return glm::vec3(x, height, z);
}

glm::vec3 circleDirection(float t) {
    //derv. of circle
    float dx = -sin(t);
    float dz = cos(t);
    glm::vec3 dir(dx, 0.0f, dz);
    return glm::normalize(dir);
        }
```

- Static, Linear, and Circular paths toggleable via keyboard input:

[1-3] Toggle Path Mode: Static

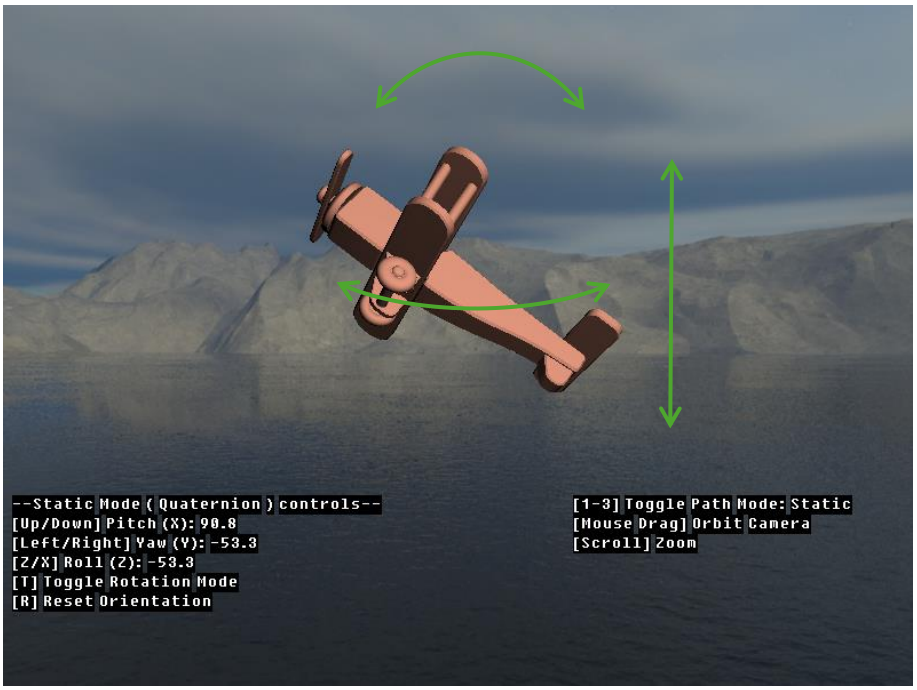
[1-3] Toggle Path Mode: Linear

[1-3] Toggle Path Mode: Circle

```
enum class PathMode {
    Static = 0,
    Linear,
    Circle
};

PathMode pathMode = PathMode::Static;
float animationTime = 0.0f;
(…)

void key_callback(GLFWwindow* window, int key, int scancode, int action, int mods) {
            (…)
                if (key == GLFW_KEY_1 && action == GLFW_PRESS)
                    pathMode = PathMode::Static;
                if (key == GLFW_KEY_2 && action == GLFW_PRESS) {
                    pathMode = PathMode::Linear;
                    animationTime = 0.0f;
                }
                if (key == GLFW_KEY_3 && action == GLFW_PRESS) {
                    pathMode = PathMode::Circle;
                    animationTime = 0.0f;
                }
}
```

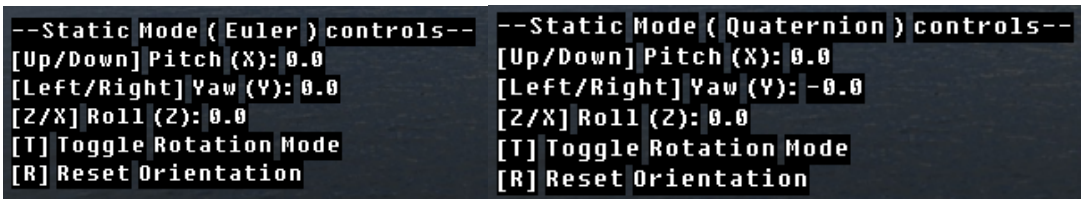
Extra Features

- Quaternion-based rotation implemented to resolve gimbal lock:



```
glm::quat orientationQuat = glm::quat(1, 0, 0, 0);

glm::vec3 quatToEulerDegrees(const glm::quat& q) {
    glm::vec3 euler = glm::degrees(glm::eulerAngles(q));
    return euler;
}

glm::quat quatFromEulerDelta(float pitchDeg, float yawDeg, float rollDeg) {
    glm::quat qPitch = glm::angleAxis(glm::radians(pitchDeg), glm::vec3(1, 0, 0));
    glm::quat qYaw = glm::angleAxis(glm::radians(yawDeg), glm::vec3(0, 1, 0));
    glm::quat qRoll = glm::angleAxis(glm::radians(rollDeg), glm::vec3(0, 0, 1));
    return qYaw * qPitch * qRoll;
```

- Rotation modes (Euler & quaternion) toggleable via keyboard input:



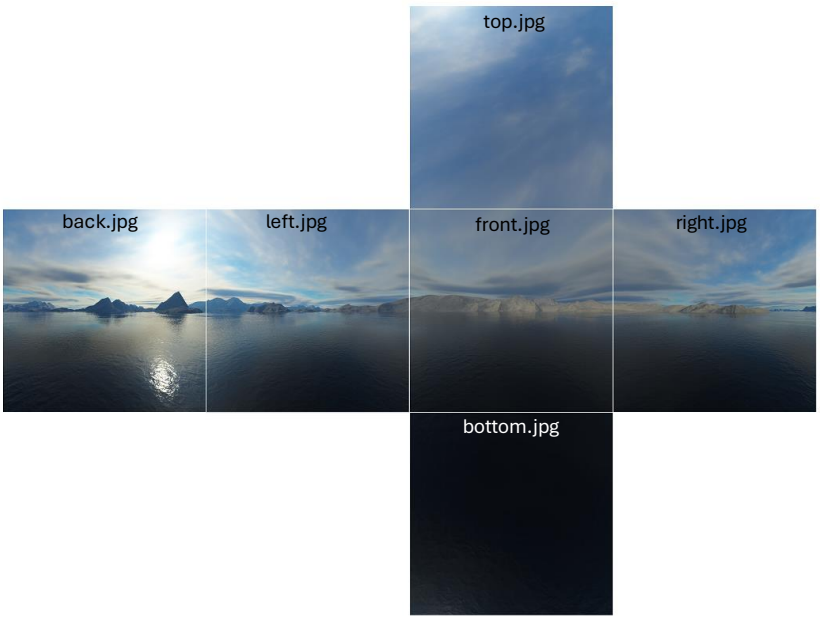

```
enum class RotationMode {
    Euler = 0,
    Quaternion
};
RotationMode rotationMode = RotationMode::Euler;

(…)

void key_callback(GLFWwindow* window, int key, int scancode, int action, int mods) {

    (…)

    if (rotationMode == RotationMode::Euler) {
        pitch += dPitch;
        yaw += dYaw;
        roll += dRoll;
    }
    else {
        glm::quat dq = quatFromEulerDelta(dPitch, dYaw, dRoll);
        orientationQuat = glm::normalize(dq * orientationQuat);
    }

    if (key == GLFW_KEY_R && action == GLFW_PRESS) {
        pitch = yaw = roll = 0.0f;
        orientationQuat = glm::quat(1, 0, 0, 0);
    }

    if (key == GLFW_KEY_T && action == GLFW_PRESS) {
        if (rotationMode == RotationMode::Euler)
            rotationMode = RotationMode::Quaternion;
        else
            rotationMode = RotationMode::Euler;

        pitch = yaw = roll = 0.0f;
        orientationQuat = glm::quat(1, 0, 0, 0);
    }

    (…)
}
```

- Linear interpolation (LERP) used to interpolate the planes position between keyframes along the linear flight path specifically:

```
glm::vec3 interpolatePosition(float t) {
    for (size_t i = 0; i < flightPath.size() - 1; i++) {
        const Keyframe& k0 = flightPath[i];
        const Keyframe& k1 = flightPath[i + 1];
```

```
        if (t >= k0.time && t <= k1.time) {
            float localT = (t - k0.time) / (k1.time - k0.time);
            return glm::mix(k0.position, k1.position, localT);
        }
    }
    return flightPath.back().position;
}
```

- Skybox added for further realism (skybox.cpp & skybox.h reused from a previous computer graphics assignment):



(Skybox Source: https://learnopengl.com/img/textures/skybox.zip)

```
#define STB_IMAGE_IMPLEMENTATION
#include <stb/stb_image.h>
#include "skybox.h"

Skybox* skybox = nullptr;
Shader* skyboxShader = nullptr;
(…)
    modelShader->setInt("skybox", 0);

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, skybox->getCubemapID());
    (…)
skybox->Draw(*skyboxShader, glmView, glmProj);
(…)
    std::vector<std::string> faces = {
        "skybox/right.jpg",
        "skybox/left.jpg",
        "skybox/top.jpg",
        "skybox/bottom.jpg",
        "skybox/front.jpg",
        "skybox/back.jpg"
    };

    skybox = new Skybox(faces);
(…)
    skyboxShader = new Shader(
        "shaders/skybox_vertex.txt",
        "shaders/skybox_fragment.txt"
    );
```

- Camera controls (zoom & orbit) via mouse input added to view scene from multiple angles:

```
#include "camera.h"
(…)
float lastX = 400.0f;
float lastY = 300.0f;
bool firstMouse = true;
bool mousePressed = false;
float cameraRadius = 25.0f;         // distance from object

Camera camera(
    glm::vec3(0.0f, 0.0f, 10.0f),   // initial position
    glm::vec3(0.0f, 1.0f, 0.0f),    // up
    -90.0f,                         // yaw
    0.0f                            // pitch
);
(…)
void mouse_button_callback(GLFWwindow* window, int button, int action, int mods) {
    if (button == GLFW_MOUSE_BUTTON_LEFT) {
        if (action == GLFW_PRESS) {
            mousePressed = true;
            firstMouse = true;
        }
        else if (action == GLFW_RELEASE) {
            mousePressed = false;
        }
    }
}

void mouse_callback(GLFWwindow* window, double xpos, double ypos) {
    if (!mousePressed) return;

    if (firstMouse) {
        lastX = (float)xpos;
        lastY = (float)ypos;
        firstMouse = false;
    }

    float xoffset = (float)xpos - lastX;
    float yoffset = lastY - (float)ypos; // reversed Y

    lastX = (float)xpos;
    lastY = (float)ypos;

    camera.ProcessMouseMovement(xoffset, yoffset);
}

void scroll_callback(GLFWwindow* window, double xoffset, double yoffset) {
    cameraRadius -= (float)yoffset;
    cameraRadius = glm::clamp(cameraRadius, 2.0f, 300.0f);
}
```